Slice B — FHIR Client + Deterministic GameContext Mapper

Owner: Malek (Bassel support) · Branch: `feature/fhir-mapper` · Merge order: second (C → B → A)

Goal: read sandbox Patient / Condition / medication resources, convert them — with plain, deterministic TypeScript — into the **same shapes the screens already consume**, and expose them through one seam so the UI never changes. The LLM is not involved (ADR-005).

---

1. Design exploration

Decision 1 — the seam (mock vs. real behind one function)

The mock data file already declares shapes "intentionally close to the future `GameContext`." We formalize that as a `GameContext` type and a `dataSource` that returns either the mock-derived or FHIR-derived value. Screens import only `getGameContext()`.

Option	Verdict
Edit each screen to branch mock/real	Rejected — spreads the decision across files, breaks "keep structure"
One `dataSource.ts` provider + `GameContext` type	**Chosen** — single swap point, screens untouched

Decision 2 — medication resource (open question #1)

`MedicationRequest` (orders) vs `MedicationStatement` (what the patient reports taking). Until the client answers: - Fetch **both**, prefer whichever returns entries, and normalize both into the same `medications[]` shape. The mapper hides the difference from the UI.

Decision 3 — fixtures-first development

Don't block the mapper on a live token. Save real sandbox JSON bundles into `src/domain/__fixtures__/` and write the mapper + tests against them. This makes the mapper: - testable in CI with no network/auth, - reproducible across the team (everyone maps the same bytes), - the artifact that resolves open question #1 (the fixtures show which resource the sandbox returns).

Decision 4 — keep `maggieMockData` as the default

`dataSource` returns mock until `AuthContext.status === 'connected'`. This keeps the offline demo working and lets Slice B merge before Slice A is finished.

---

2. Files this slice adds

```
apps/mobile/src/
├── domain/
│   ├── gameContext.ts        # GameContext type + fromMock() adapter
│   ├── fhirMapper.ts         # FHIR bundles → GameContext (pure, deterministic)
│   ├── dataSource.ts         # getGameContext(): mock | mapped real
│   └── __fixtures__/
│       ├── patient.json
│       ├── conditions.json
│       └── medications.json
└── services/
    └── fhir.ts               # typed reads against EXPO_PUBLIC_EPIC_FHIR_BASE_URL
```

The only edits to existing files are screens importing `getGameContext()` instead of importing the mock module directly (one import line each), done additively.

---

3. The `GameContext` type

A superset of what the mock exposes, so the mapper and the mock both satisfy it. Aligns with the `GameContext` sketch in `docs/architecture.md`, extended for the existing Demo-1 screens.

```
// src/domain/gameContext.ts
export type Confidence = 'sandbox-record' | 'maggie-believes';

export interface GameContext {
  source: 'SYNTHETIC_SANDBOX_DEMO' | 'EPIC_SANDBOX';
  patientDisplayName: string;
  conditions: { id: string; label: string; note?: string; confidence: Confidence }[];
  devices:    { id: string; label: string; kind: string; confidence: Confidence }[];
  medications:{ id: string; name: string; generalUse?: string;
                relatedConditionId?: string; confidence: Confidence }[];
  // Beliefs and symptom sets stay app-authored in P1 (not in FHIR); carried through unchanged.
  beliefs: { id: string; maggieBelieves: string; checkWith: string }[];
}
```

```
// fromMock(): wraps the existing mock module as a GameContext (no UI change)
import * as mock from './maggieMockData';
export function fromMock(): GameContext {
  return {
    source: 'SYNTHETIC_SANDBOX_DEMO',
    patientDisplayName: mock.maggieProfile.displayName,
    conditions: mock.conditions,
    devices: mock.devices,
    medications: mock.medications,
    beliefs: mock.beliefs,
  };
}
```

> Note: `beliefs` and the symptom/quiz content are **app-authored**, not clinical FHIR facts, and deliberately stay out of mapped data (mirrors the "Maggie believes" framing and ADR-006's gameplay/clinical separation). The mapper only fills `conditions`, `devices`, `medications`, and `patientDisplayName`.

---

4. `fhir.ts` — typed reads

```
import Constants from 'expo-constants';
const FHIR_BASE = String(Constants.expoConfig?.extra?.epicFhirBaseUrl ?? '');

async function read(path: string, accessToken: string) {
  const res = await fetch(`${FHIR_BASE}/${path}`, {
    headers: { Authorization: `Bearer ${accessToken}`, Accept: 'application/fhir+json' },
  });
  if (!res.ok) throw new Error(`FHIR ${path} → ${res.status}`);
  return res.json(); // a FHIR Bundle or resource
}

export async function fetchPatient(id: string, token: string)    { return read(`Patient/${id}`, toke
export async function fetchConditions(pid: string, token: string){ return read(`Condition?patient=${
export async function fetchMedications(pid: string, token: string) {
  // Open question #1: try both, prefer non-empty.
  const reqs = await read(`MedicationRequest?patient=${pid}`, token);
  if (reqs?.entry?.length) return { kind: 'MedicationRequest', bundle: reqs } as const;
  const stmts = await read(`MedicationStatement?patient=${pid}`, token);
  return { kind: 'MedicationStatement', bundle: stmts } as const;
}
```

- `accessToken` and the launch `patientId` come from Slice A's `AuthContext`.
- No token logging. Errors surface a friendly message via the existing card UI; raw bodies are not rendered (they could contain bundle detail).

---

5. `fhirMapper.ts` — deterministic mapping

Pure functions; no `Date.now()`, no randomness, no network — same input always yields same output (this is what makes it unit-testable and ADR-005 compliant).

```
import { GameContext, Confidence } from './gameContext';

const SANDBOX: Confidence = 'sandbox-record';

export function mapConditions(bundle: any): GameContext['conditions'] {
  return (bundle.entry ?? []).map((e: any, i: number) => ({
    id: e.resource?.id ?? `cond-${i}`,
    label: e.resource?.code?.text
        ?? e.resource?.code?.coding?.[0]?.display
        ?? 'Unknown condition',
    confidence: SANDBOX,
  }));
}

export function mapMedications(kind: string, bundle: any): GameContext['medications'] {
  return (bundle.entry ?? []).map((e: any, i: number) => {
    const r = e.resource ?? {};
    const cc = r.medicationCodeableConcept;
    return {
      id: r.id ?? `med-${i}`,
      name: cc?.text ?? cc?.coding?.[0]?.display ?? 'Unknown medication',
      confidence: SANDBOX,
    };
  });
}

export function toGameContext(input: {
  patient: any; conditions: any; meds: { kind: string; bundle: any };
}): GameContext {
  return {
    source: 'EPIC_SANDBOX',
    patientDisplayName: nameOf(input.patient),     // HumanName → "Given Family"
    conditions: mapConditions(input.conditions),
    devices: [],                                   // Device fetch optional in P1
    medications: mapMedications(input.meds.kind, input.meds.bundle),
    beliefs: [],                                   // app-authored, not from FHIR
  };
}
```

- `nameOf()` reads `Patient.name[0]` (`given` + `family`), falling back to `'Patient'`.
- **Devices** (Dexcom/Omnipod in the mock) come from FHIR `Device` / `DeviceUseStatement` — optional for P1's Definition of Done (meds + conditions). Leave `devices: []` or add a `mapDevices()` if the sandbox patient has them; the Patient Snapshot screen already tolerates empty arrays.

---

6. `dataSource.ts` — the runtime swap

```
import { GameContext } from './gameContext';
import { fromMock } from './gameContext';
import * as fhir from '../services/fhir';
import { toGameContext } from './fhirMapper';

export async function getGameContext(auth: {
  status: string; patientId?: string; getAccessToken: () => Promise<string | null>;
}): Promise<GameContext> {
  if (auth.status !== 'connected' || !auth.patientId) return fromMock();   // default path
  const token = await auth.getAccessToken();
  if (!token) return fromMock();
  const [patient, conditions, meds] = await Promise.all([
    fhir.fetchPatient(auth.patientId, token),
    fhir.fetchConditions(auth.patientId, token),
    fhir.fetchMedications(auth.patientId, token),
  ]);
  return toGameContext({ patient, conditions, meds });
}
```

Wiring the Patient Snapshot screen (the one visible P1 change)

`app/patient-snapshot.tsx` currently imports `conditions`, `devices`, `medications` from the mock. Change it to:

```
const auth = useAuth();
const [ctx, setCtx] = useState<GameContext>(() => fromMock()); // instant mock render
useEffect(() => { getGameContext(auth).then(setCtx); }, [auth.status]);
```

Render `ctx.conditions` / `ctx.medications`. When disconnected it shows the mock (offline demo); when connected it shows the real sandbox patient. The `DemoBadge` reads `ctx.source` to label "synthetic" vs "Epic sandbox." Medication Match keeps using mock quiz content in P1 (quiz generation is P2) — only the snapshot reads real data.

---

7. Tests (deterministic, no network)

Add a test runner — `jest-expo` is the SDK-aligned choice (`bunx expo install jest-expo jest @types/jest`), or `bun test` for plain TS unit tests of the mapper.

```
// fhirMapper.test.ts
import conditions from './__fixtures__/conditions.json';
import { mapConditions } from './fhirMapper';

test('maps condition display text deterministically', () => {
  const out = mapConditions(conditions);
  expect(out.length).toBeGreaterThan(0);
  expect(out[0].label).not.toBe('Unknown condition');
  expect(out).toEqual(mapConditions(conditions)); // pure: same in = same out
});
```

Fixtures are real sandbox bundles with **no real PHI** (Epic sandbox patients are synthetic) — safe to commit, and they document exactly which resource shapes the mapper handles.

---

8. Guardrails / review checklist (Slice B)

- [ ] Mapper is pure: no `Date.now`, `Math.random`, network, or `console.log` of bundles.
- [ ] Screens import `getGameContext` / `GameContext` only — no screen branches on auth itself.
- [ ] `beliefs` / quiz / symptom content stay app-authored (not derived from FHIR).
- [ ] No raw FHIR bundle rendered to the UI or logged.
- [ ] Fixtures are sandbox-synthetic; confirmed no real identifiers.
- [ ] Empty arrays handled (patient with no meds/conditions doesn't crash the snapshot).

---

9. Commit sequence (feature/fhir-mapper)

#	Commit message	Contents	Push?
1	`feat(domain): add GameContext type + fromMock adapter`	`gameContext.ts`	Push → open **draft PR**
2	`feat(domain): dataSource seam (mock default)`	`dataSource.ts` returning mock	Push
3	`refactor(ui): patient-snapshot reads getGameContext`	screen edit, still mock	Push
4	`test: add sandbox FHIR fixtures`	`__fixtures__/*.json`	Push
5	`feat(services): typed FHIR reads`	`fhir.ts`	Push
6	`feat(domain): deterministic FHIR → GameContext mapper`	`fhirMapper.ts`	Push
7	`test(domain): mapper unit tests on fixtures`	`fhirMapper.test.ts`	Push
8	`feat: wire real path in dataSource behind auth status`	connect mapper to seam	Push → **ready for review**

Push on commit 1 (draft PR so CI runs the mapper tests early). **Request review** after commit 8 when fixtures + tests are green. Note commits 1–3 are safe to merge even before the real path exists (default stays mock) — useful if you want to land the seam first.

---

10. Definition of done (Slice B)

- `GameContext` type exists; `fromMock()` and `toGameContext()` both satisfy it.
- Mapper unit tests pass on committed sandbox fixtures.

- Patient Snapshot shows mock when disconnected and **real sandbox** Patient/Condition/medication data when Slice A reports `connected`.
- No raw FHIR reaches the UI, logs, or git; mapper is pure.
- Provides `getGameContext()` that Slice C records and that P2's `generate-quiz` will consume.
